WEBRUM-08: Dashboards and Alerting

> **Series:** WEBRUM — Web Real User Monitoring | **Notebook:** 8 of 10 | **Created:** March 2026 | **Last Updated:** 10/14/2026

Overview

An effective RUM monitoring strategy requires both visibility (dashboards) and proactive detection (alerting). Dashboards provide at-a-glance health views for different audiences — executives care about business impact and Apdex trends, while operations teams need real-time error rates and performance breakdowns.

Table of Contents

1. Executive RUM Dashboard KPIs
2. Apdex Score Calculation
3. Operational RUM Dashboard
4. RUM Alerting Strategies
5. Error Rate Alerts
6. Performance Degradation Alerts
7. RUM + Synthetic Combined View
8. Summary and Series Recap

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail enabled
RUM Enabled	Web applications with at least 7 days of data
Permissions	`storage:events:read`, `storage:metrics:read`, dashboard create permissions
Previous Notebooks	All prior WEBRUM notebooks (01-07)

1. Executive RUM Dashboard KPIs

Executive dashboards should be simple, business-focused, and actionable. Key metrics:

KPI	What It Tells Executives	Target
Apdex score	Overall user satisfaction (0-1)	> 0.85
Session count	Traffic volume and trends	Depends on business
Error rate %	Percentage of sessions with errors	< 2%
Bounce rate %	Users leaving after one page	< 40%
CWV pass rate	% of page loads meeting all 3 CWV	> 75%
Avg session duration	User engagement level	Varies by app type

```
// Session/performance field vocabulary corrected 08/12/2026 (New RUM).
Classic camelCase RUM
// names are null on New RUM data and fail silently. Verified against 3,261
user.sessions:
//   userType -&gt; dt.rum.user_type      userActionCount -&gt;
user_action_count
//   totalErrorCount -&gt; error.count    sessionId -&gt; dt.rum.session.id
//   hasSessionReplay -&gt; characteristics.has_replay
//   browserFamily -&gt; browser.name     osFamily -&gt; os.name
//   application -&gt; frontend.name (stable; an array on user.sessions, so
expand it before grouping)
//                  primary_tags.application exists only if your OpenPipeline
adds it (corrected 10/14/2026)
//   country/city/continent -&gt; geo.country.name / geo.city.name /
geo.continent.name
//   screen.width|height -&gt; browser.window.width|height
//   dom.interactive.time -&gt; performance.dom_interactive
//   load.event.time -&gt; performance.load_event_end
//   server.time -&gt; ttfb.value (page summaries;
web_vitals.time_to_first_byte where populated) —
//                  NOT ttfb.waiting_duration, which is the pre-request
wait/redirect phase
// TWO TENANT CAVEATS on the validation tenant, both of which leave a CORRECT
query empty:
//   * every session is dt.rum.user_type == "synthetic", so a real-user filter
matches nothing —
//     the documented values are "real_user" / "robot" / "synthetic"
(lowercase);
//   * geo.* is 0-populated, because synthetic traffic carries no geolocation.
// Executive KPI summary — single-query dashboard tile
fetch user.sessions, from:-24h
| filter dt.rum.user_type == "real_user"
| expand frontend.name   // array on sessions: a multi-frontend session counts
once per frontend
| summarize
    {total_sessions = count(),
    error_sessions = countIf(error.count &gt; 0),
    bounce_sessions = countIf(user_action_count == 1),
    avg_actions = avg(user_action_count),
    avg_duration_min = avg(duration / 1m)},
    by:{frontend.name}
| fieldsAdd error_rate_pct = round(toDouble(error_sessions) /
toDouble(total_sessions) * 100.0, decimals: 1),
    bounce_rate_pct = round(toDouble(bounce_sessions) /
toDouble(total_sessions) * 100.0, decimals: 1),
    avg_duration_min = round(avg_duration_min, decimals: 1)
| sort total_sessions desc
```

```
// Session volume trend — 7-day daily session counts
fetch user.sessions, from:-7d
| filter dt.rum.user_type == "real_user"
| expand frontend.name   // array on sessions: a multi-frontend session counts
once per frontend
| makeTimeseries session_count = count(), interval:24h, by:{frontend.name}
```

2. Apdex Score Calculation

The **Application Performance Index (Apdex)** is an open standard for measuring user satisfaction. It classifies every user action into three buckets based on a threshold T:

Classification	Duration	Description
Satisfied	≤ T	Action completed within acceptable time
Tolerating	T < duration ≤ 4T	Action was slow but user waited
Frustrated	> 4T	Action was too slow or failed

Apdex formula:

```
Apdex = (Satisfied + (Tolerating / 2)) / Total
```

The threshold T is configurable per application. Common defaults:

Action Type	Default T
Page load	3 seconds
XHR action	2.5 seconds
Route change	2.5 seconds

Classify page loads on navigation events. The Apdex, SLA and page-load alert queries below read `performance.load_event_end` on events with `characteristics.has_w3c_navigation_timings == true` — the field and filter Dynatrace's own page-load query uses. A page summary's `duration` is how long the page was open, not how long it took to load: a page summary covers the page instance until *"the next document request starts or when the tab is closed"*. Apdex built on it scores reading time. On the synthetic-only validation tenant (24 h, 10/05/2026) the same T = 3 s Apdex read 0.99 on load time and 0.32 on page-summary duration.

> **Sources:** Monitor web performance with DQL (DT docs) — *"Load event end measures the time from navigation start to the completion of the browser's load event."*; Navigation-related events — semantic dictionary (DT docs) — *"A page instance begins with each hard navigation and ends when the next document request starts or when the tab is closed."*

```
// Page-load time corrected 10/05/2026: performance.load_event_end on
navigation events
// (has_w3c_navigation_timings). A page summary's `duration` is how long the
page was OPEN.
// Apdex calculation with T = 3 seconds for page loads
fetch user.events, from:-24h
| filter characteristics.has_w3c_navigation_timings == true
| fieldsAdd load_sec = performance.load_event_end / 1s
| summarize {total = count(),
    satisfied = countIf(load_sec &lt;= 3),
    tolerating = countIf(load_sec &gt; 3 and load_sec &lt;= 12),
    frustrated = countIf(load_sec &gt; 12)},
    by:{frontend.name}
| fieldsAdd apdex = round((toDouble(satisfied) + toDouble(tolerating) / 2.0) /
toDouble(total), decimals: 3)
| sort apdex asc
```

```
// Page-load time corrected 10/05/2026: performance.load_event_end on
navigation events
// (has_w3c_navigation_timings). A page summary's `duration` is how long the
page was OPEN.
// Apdex trend over 7 days — daily Apdex score (tolerating is pre-halved in
is_tolerating)
fetch user.events, from:-7d
| filter characteristics.has_w3c_navigation_timings == true
| fieldsAdd load_sec = performance.load_event_end / 1s
| fieldsAdd is_satisfied = if(load_sec &lt;= 3, 1.0, else: 0.0),
    is_tolerating = if(load_sec &gt; 3 and load_sec &lt;= 12, 0.5, else: 0.0)
| makeTimeseries {
    total_actions = count(),
    satisfied_score = sum(is_satisfied),
    tolerating_score = sum(is_tolerating)},
    interval:24h
| fieldsAdd apdex = (satisfied_score[] + tolerating_score[]) / total_actions[]
```

```
// Page-load time corrected 10/05/2026: performance.load_event_end on
navigation events
// (has_w3c_navigation_timings). A page summary's `duration` is how long the
page was OPEN.
// Apdex by page — which pages have the worst user satisfaction?
fetch user.events, from:-24h
| filter characteristics.has_w3c_navigation_timings == true
| fieldsAdd load_sec = performance.load_event_end / 1s
| summarize {total = count(),
    satisfied = countIf(load_sec &lt;= 3),
    tolerating = countIf(load_sec &gt; 3 and load_sec &lt;= 12)},
    by:{page.detected_name}
| filter total &gt; 20
| fieldsAdd apdex = round((toDouble(satisfied) + toDouble(tolerating) / 2.0) /
toDouble(total), decimals: 3)
| sort apdex asc
| limit 10
```

Interpreting Apdex Scores

Apdex Range	Rating	Action
0.94 - 1.00	Excellent	No action needed
0.85 - 0.93	Good	Monitor for changes
0.70 - 0.84	Fair	Investigate slow pages
0.50 - 0.69	Poor	Prioritize optimization
< 0.50	Unacceptable	Immediate attention required

3. Operational RUM Dashboard

Operations teams need real-time visibility into errors, performance anomalies, and traffic patterns.

```
// Error / navigation vocabulary corrected 08/12/2026 (New RUM):
//   filter type == "Error"  -&gt; filter characteristics.has_error == true
(11,909 events; identical
//                              population to isNotNull(error.type), whose
values are request/csp/exception)
//   error.message           -&gt; error.reason
//   user_action.type: "Load" -&gt; "hard_navigation", "RouteChange" -&gt;
"soft_navigation",
//                     "Xhr" -&gt; "same_view", "Custom" -&gt; "api"
(documented values, corrected 10/05/2026)
//   connection.type         -&gt; no web RUM equivalent
(network.connection.type is OneAgent for Mobile
//                              only; network.protocol.name is the OSI
protocol, e.g. "http")
// THE EVENT CHARACTERISTIC MATTERS AS MUCH AS THE FIELD: navigation-timing
fields
// (performance.dom_interactive, performance.load_event_end) live on
navigation events
// (characteristics.has_navigation) and are unpopulated on page summaries
// (characteristics.has_page_summary), so a page-summary filter silently
empties them.
// ttfb.* is the opposite — it lives on page summaries. Select with the stable
has_* flags, not
// characteristics.classifier (corrected 09/28/2026): the docs call classifier
"not intended for
// query usage" and Semantic Dictionary 1.349 removes it from the user-event
models.
// Real-time error rate — errors per 15 minutes over the last 6 hours
fetch user.events, from:-6h
| filter characteristics.has_error == true
| makeTimeseries error_count = count(), interval:15m, by:{error.type}
```

```
// Active error summary — current top errors in the last hour
fetch user.events, from:-1h
| filter characteristics.has_error == true
| summarize {error_count = count(),
    affected_sessions = countDistinct(dt.rum.session.id)},
    by:{error.reason, error.type, frontend.name}
| sort affected_sessions desc
| limit 10
```

```
// Page-load time corrected 10/05/2026: performance.load_event_end on
navigation events
// (has_w3c_navigation_timings). A page summary's `duration` is how long the
page was OPEN.
// Performance SLA — percentage of page loads under 3 seconds
fetch user.events, from:-1h
| filter characteristics.has_w3c_navigation_timings == true
| fieldsAdd load_sec = performance.load_event_end / 1s
| summarize {total = count(),
    under_3s = countIf(load_sec &lt;= 3)},
    by:{frontend.name}
| fieldsAdd sla_pct = round(toDouble(under_3s) / toDouble(total) * 100.0,
decimals: 1)
| sort sla_pct asc
```

4. RUM Alerting Strategies

Effective RUM alerting requires the right balance — too sensitive triggers alert fatigue, too loose misses real issues.

Recommended Alert Categories

Category	What to Alert On	Threshold Example
Error rate spike	Sudden increase in JS error rate	> 5% of sessions with errors
Apdex drop	User satisfaction below threshold	Apdex < 0.7 for 15 minutes
Performance degradation	p75 page load exceeds baseline	p75 > 5s for 15 minutes
Traffic anomaly	Unexpected drop in session volume	< 50% of previous day's hourly average
CWV regression	Core Web Vitals crossing into Poor	LCP p75 > 4s for 30 minutes

Alert Configuration

Modern path (recommended for new alerting): build these conditions as **Davis anomaly detectors** (`builtin:davis.anomaly-detectors`), driven by DQL — configured in Settings from SaaS 1.344, or in the Anomaly Detection app on earlier versions (see the callout below). Detectors read Grail, so they work against the New RUM field vocabulary the rest of this series uses, and they carry forward past the upgrade.

> **Where detectors are created (SaaS 1.344).** *"Starting with Dynatrace version 1.344, custom alerts have moved to Settings. Because Anomaly Detection is deprecated, we highly recommend that you use Settings to access your existing configurations and create new ones."* SaaS 1.344 rolls out to tenants in stages — check your tenant's version before following either path. On earlier versions the **Anomaly Detection** app is where custom alerts are created, and the modern path above is built there.
>
> **Sources:** Anomaly Detection (DT docs) — *"Starting with Dynatrace version 1.344, custom alerts have moved to Settings."*

Legacy path — metric events: the classic surface below still works and is what most existing RUM alerting uses, but `builtin:anomaly-detection.metric-events` is flagged **Blocked at upgrade**, and the `builtin:apps.web.*` metric keys it alerts on are Metrics Classic keys that **DQL cannot query** (FAQ entry 11 §3.1). Both facts point the same way: do not author new RUM alerting here.

Configure the legacy path via:

Settings > Anomaly detection > Metric events > Add metric event

Setting	Recommended Value
Metric	RUM metric (e.g., `builtin:apps.web.action.apdex`)
Evaluation window	15 minutes (balances speed and noise)
Sliding window	5-minute intervals
Threshold type	Static or auto-adaptive baseline

5. Error Rate Alerts

The following queries provide the data foundation for error rate alerting. Use these patterns to detect when error rates exceed acceptable thresholds.

```
// Error rate per 15-minute window — alert threshold data
fetch user.sessions, from:-6h
| filter dt.rum.user_type == "real_user"
| expand frontend.name   // array on sessions: a multi-frontend session counts
once per frontend
| fieldsAdd has_error = if(error.count &gt; 0, 1.0, else: 0.0)
| makeTimeseries
    {total_sessions = count(),
    error_sessions = sum(has_error)},
    interval:15m,
    by:{frontend.name}
```

```
// New errors — error groups first seen in the last hour (potential deployment
issue).
// Corrected 10/05/2026: user.events has no populated timestamp (use
start_time), and "first seen"
// must be measured over a longer lookback than the window it is compared
against — with
// from:-1h every recurring error looked new.
fetch user.events, from:-7d
| filter characteristics.has_error == true
| summarize {first_seen = min(start_time), error_count = count(),
    affected_sessions = countDistinct(dt.rum.session.id)},
    by:{error.reason, error.type}
| filter first_seen &gt; now() - 1h
| sort first_seen desc
| limit 10
```

6. Performance Degradation Alerts

Detect when page load performance degrades beyond acceptable thresholds.

```
// Page-load time corrected 10/05/2026: performance.load_event_end on
navigation events
// (has_w3c_navigation_timings). A page summary's `duration` is how long the
page was OPEN.
// p75 page load time per 15-minute window — performance alert data
fetch user.events, from:-6h
| filter characteristics.has_w3c_navigation_timings == true
| fieldsAdd load_ms = performance.load_event_end / 1ms
| makeTimeseries p75_load_ms = percentile(load_ms, 75), interval:15m, by:
{frontend.name}
```

```
// Page-load time corrected 10/05/2026: performance.load_event_end on
navigation events
// (has_w3c_navigation_timings). A page summary's `duration` is how long the
page was OPEN.
// Apdex per 15-minute window — satisfaction alert data
fetch user.events, from:-6h
| filter characteristics.has_w3c_navigation_timings == true
| fieldsAdd load_sec = performance.load_event_end / 1s
| fieldsAdd apdex_score = if(load_sec &lt;= 3, 1.0,
    else: if(load_sec &lt;= 12, 0.5,
    else: 0.0))
| makeTimeseries avg_apdex = avg(apdex_score), interval:15m, by:
{frontend.name}
```

> **Tip:** For production alerting, use Dynatrace Intelligence anomaly detection rather than static thresholds. Dynatrace Intelligence automatically baselines normal behavior and detects deviations, reducing false positives from seasonal traffic patterns.

7. RUM + Synthetic Combined View

RUM and synthetic monitoring complement each other. Combining them provides a complete UX monitoring picture:

Aspect	RUM Covers	Synthetic Covers
Availability	Only when users visit	24/7 scheduled monitoring
Baseline	Real-world conditions (variable)	Clean-room conditions (stable)

Aspect	RUM Covers	Synthetic Covers
Coverage	Pages users visit	All critical paths (scripted)
Error detection	Errors users hit	Errors in scripted flows
Geographic	Where users are	From configured locations

Side-by-Side Comparison

Use `append` to compare RUM and synthetic performance for the same application:

```
// RUM session summary — real user experience
fetch user.sessions, from:-24h
| filter dt.rum.user_type == "real_user"
| expand frontend.name   // array on sessions: a multi-frontend session counts
once per frontend
| summarize {rum_sessions = count(),
    rum_error_sessions = countIf(error.count &gt; 0),
    rum_avg_actions = avg(user_action_count)},
    by:{frontend.name}
| fieldsAdd monitoring_type = "RUM",
    error_rate_pct = round(toDouble(rum_error_sessions) /
toDouble(rum_sessions) * 100.0, decimals: 1)
```

```
// Synthetic vs RUM — compare session types for the same app (user-type values
are lowercase:
// "real_user", "robot", "synthetic"; "SYNTHETIC" matched nothing — corrected
10/05/2026)
fetch user.sessions, from:-24h
| expand frontend.name   // array on sessions: a multi-frontend session counts
once per frontend
| summarize {session_count = count(),
    avg_duration_sec = avg(duration / 1s),
    error_sessions = countIf(error.count &gt; 0)},
    by:{frontend.name, dt.rum.user_type}
| filter dt.rum.user_type == "real_user" or dt.rum.user_type == "synthetic"
| fieldsAdd error_rate_pct = round(toDouble(error_sessions) /
toDouble(session_count) * 100.0, decimals: 1)
| sort frontend.name asc, dt.rum.user_type asc
```

8. Summary and Series Recap

In this notebook, we covered:

- **Executive KPIs** — Session count, error rate, bounce rate, Apdex
- **Apdex calculation** — Satisfied/tolerating/frustrated classification via DQL
- **Operational dashboard** — Real-time error rates, SLA tracking, active errors
- **Alerting strategies** — Error rate spikes, Apdex drops, performance degradation
- **RUM + Synthetic** — Combining both monitoring types for complete coverage

WEBRUM Series Recap

Notebook	Topic	Key Takeaway
01	RUM Fundamentals	JavaScript agent, data model, basic queries
02	SPA Instrumentation	Route changes, framework tips, XHR monitoring
03	Core Web Vitals	LCP, INP, CLS measurement and scoring
04	Session Analysis	Segmentation, journeys, conversions, bounce rates
05	Error Analysis	Error impact, XHR errors, error-session correlation
06	Performance Analysis	Waterfall, TTFB, slow pages, geographic performance
07	Session Replay	Privacy, masking, finding and correlating replays
08	Dashboards & Alerting	Apdex, KPI dashboards, alerting strategies

References

- Apdex Standard
- Dynatrace RUM Dashboards
- Metric Events for Alerting
- User events — semantic dictionary (DT docs) — *"Used for internal optimization when storing the data and not intended for query usage."*
- Semantic Dictionary changelog 1.349 (DT docs)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.